# The Container Model: An Experience Report on Enforcing Standards Across Human–AI Software Units

Zhongqiang Fu

August 2026

## Abstract

AI coding assistants now let every role in a small organization — including people who have never used a terminal — own a complete software system. The risk this creates is organizational, not technical: each AI, optimizing locally, invents its own conventions, leaving systems that cannot be governed, upgraded, or federated. We describe the **container model**, whose unit is the **box**: a standardized, partially immutable software stack under accountable human ownership, worked by any number of humans and AI agents — for automated roles, none in the daily loop. Five requirements make the standard consequential rather than advisory: a frame that upgrades overwrite wholesale, upstream-first evolution, per-turn governance of the AIs, mediated cross-box interaction, and proven resource claims between co-located boxes. The pattern is bounded by AI capability on both sides: what makes a box affordable to fill is what makes bounding its blast radius necessary. We price its costs openly: version lag, a triage bottleneck, provisioning burden, and amplified standard errors. We then report ten weeks with one implementation — SOLO, an open-source AI-native microservice framework — across seven derived production systems: 21 tagged releases; 31 feedback reports, 23 triaged at a median of two days, accepted proposals upcycled into releases; a natural experiment in which all five observed projects independently re-invented the same missing artifact; and a provisioning case study for a non-technical owner. We distill six lessons and argue that enforced invariance — not better prompting — is the missing primitive for scaling AI-assisted development beyond one person.

**Keywords:** human–AI collaboration, AI-assisted software engineering, platform engineering, software governance, multi-agent organizations, experience report

---

## 1. Introduction

Large-language-model coding assistants have crossed a threshold: a single person working with an AI can build and operate a production software system end to end. The natural next step for a small organization is to give *every* role — finance, marketing, product — its own system, run by that role's humans and AIs, and to connect these systems later through a coordination layer.

The moment an organization tries this, it hits a problem that is neither a model-capability problem nor a prompting problem. Each AI, asked to build a system, will make hundreds of small architectural decisions: how entities are stored, how services talk to each other, how secrets are handled, what a permission is. Left alone, N locally optimizing AIs produce N mutually alien architectures. Convention documents do not prevent this — an AI (or a person) can read a style guide and still diverge, because *diverging has no consequences*. We argue this is the principal risk of scaling AI-assisted development, and that the known industrial answer to it — platform engineering's "golden paths" [1,2] — is necessary but not sufficient,

because golden paths are advisory and are designed for engineers inside one platform team's jurisdiction, not for federated units that each own their whole stack.

This paper's subject is a pattern, not a product. We first define the **container model** in implementation-free terms: its unit and boundary, the five requirements any implementation must satisfy, the trade it offers — what enforced invariance buys and what it costs — and the capability threshold past which the pattern becomes meaningful at all (§2). We then present **SOLO**, an open-source AI-native microservice framework, as one concrete implementation, mapping each requirement to the mechanism that realizes it (§3). Ten weeks of measured experience across seven derived production systems supply the evidence (§4), from which we distill six lessons (§5).

**Contributions.**

1. An implementation-independent articulation of the container model — unit, boundary, five requirements, an explicit cost/benefit ledger, and the capability threshold that makes the pattern both affordable and necessary (§2) — with a point-by-point separation from adjacent patterns (§6, Table 3).
2. An existence proof: SOLO, a complete open-source implementation, with the requirement-to-mechanism map (§3).
3. Ten weeks of measured evidence: release and feedback-loop statistics, three traced feedback-to-release cases, a 5-of-5 natural experiment revealing a gap in the standard itself, and a provisioning case study for a non-technical owner (§4).
4. Six transferable lessons, several of which contradicted our initial design intuitions (§5).

We do not claim a controlled evaluation; this is an experience report from a single small organization running a single implementation, with the threats to validity that entails (§7).

## 2. The Container Model

The pattern is named after the ISO intermodal container [3] — not after OS-level containerization, with which it shares nothing but the word. The shipping container succeeded for one reason: its specification did not bend to its users. Sixty years of invariance is what made every port, crane, ship and truck interoperable. The container model applies the same logic to organizations of humans and AIs. One clarification before the definitions: the principle this paper argues for is **enforced invariance** — a standard whose violation carries an automatic cost; the container is the metaphor that names it and, as §2.5 shows, the historical case that predicts when it pays. The metaphor is not the argument. This section defines the pattern without reference to any implementation: everything here is a definition, a requirement, or a claimed consequence; §3 supplies one implementation and §4 the measurements.

### 2.1 The unit and its boundary

The unit of organization is not a person, not an AI agent, and not a team: it is the **box** — a complete, independently deployable software stack instantiated from a shared scaffold, under accountable human ownership. Humans own purpose, judgment, approval, and accountability; AIs do construction and operation; the box is the jurisdiction they share.

Every box is split by one boundary into two zones:

- **The frame:** framework code, wire contracts, authoring rules, checkers, deployment machinery. The frame is *identical across all boxes* and is not the box's to edit.
- **The payload:** the box's own services, data, environment, and purpose documents. The payload is entirely the box's business; no central actor touches it.

Who works a box is deliberately left open: a box is an **n-human, n-AI** unit, and on the human side n may be zero. One owner conversing with one assistant is one configuration; several humans and several AI actors at once is another; a fully automated role — scheduled collection, event-driven reaction — whose daily operation involves no human at all is a third. The one count that is never zero is ownership: every box answers to an accountable human owner, even when that owner never appears in its daily operation. What the pattern fixes is not the head-count but the boundary: the box is the unit of ownership and accountability, and boxes never call each other's internals — cross-box interaction goes through a mediated coordination layer or does not happen.

### 2.2 Five requirements

A container deployment stands or falls on five properties. We state them as requirements on any implementation.

**R1 — The frame is enforced, not advised.** Violating the frame must carry an automatic cost, not a reviewer's objection. The reference mechanism is *overwrite-on-upgrade*: the frame is a designated read-only zone that every framework upgrade replaces wholesale, so a local modification to the frame is not a violation someone might catch — it is work the next upgrade deletes. A box that cannot tolerate that deletion must escalate (R2) or carry its divergence as visible, enumerated debt. A standard whose violation costs nothing is a style guide; the consequence is what makes it a standard. The enforcement must also operate at the speed the AIs work — a contract check that fires at review time is already hundreds of decisions too late.

**R2 — The standard evolves upstream-first.** Invariance without an evolution path is a straitjacket. When real work inside a box is blocked by the frame, the correct move is never a local patch: it is a structured feedback report to the standard's maintainer — source and scenario, *evidence with provenance labels* (first-hand measurement vs. second-hand citation vs. judgment), observed behavior, root cause, proposals ranked by value — triaged on the record, whether or not the proposal is accepted. Accepted changes ship in the next release *to every box*; the reporting box then deletes its workaround. This is upstream-first open-source discipline [4] transplanted to an organization's internal human–AI infrastructure — with one addition worth naming: part of the feedback should come from the AIs themselves, at the moment a task hits a wall.

**R3 — AIs are governed per-turn, before code.** The highest-impact AI failure modes occur before any file is edited: proposing features nobody asked for, creating speculative schemas "to be ready," persisting irreproducible data with no export path. Edit-triggered guardrails structurally cannot reach that window. Each box must therefore carry a governing document loaded into every AI session's context on every conversational turn — stating the box's purpose, its decision criteria, its data classes and their required treatment, and the R2 rule that frame-level friction goes upstream rather than into local patches.

**R4 — Cross-box interaction is mediated.** Boxes never call each other's internals. Federation happens through an authenticated coordination layer whose own configuration must be governed at least as strictly as any box — because that layer is, by construction, the permission-concentration point of the whole organization.

**R5 — Resource claims are proven, not assumed.** R4 governs what boxes say to each other; R5 governs what they silently share. Wherever boxes are co-located — the common case, since a box is small enough to sit beside its neighbours on one machine — every box must prove exclusive ownership of the infrastructure it claims (ports, databases, inherited environment) at startup, and fail closed when it cannot. This requirement is not a deployment detail but a precondition of the containment claim in §2.5: a box whose blast radius is bounded by mediated calls alone is not bounded at all, because the damaging path in practice is not a call — it is two boxes quietly attaching to the same database. We state it as a requirement because our own standard lacked it and paid for the omission (§4.1, L3).

Figure 1 shows where the five requirements bite, and Figure 2 the loop that keeps the frame from becoming a straitjacket.

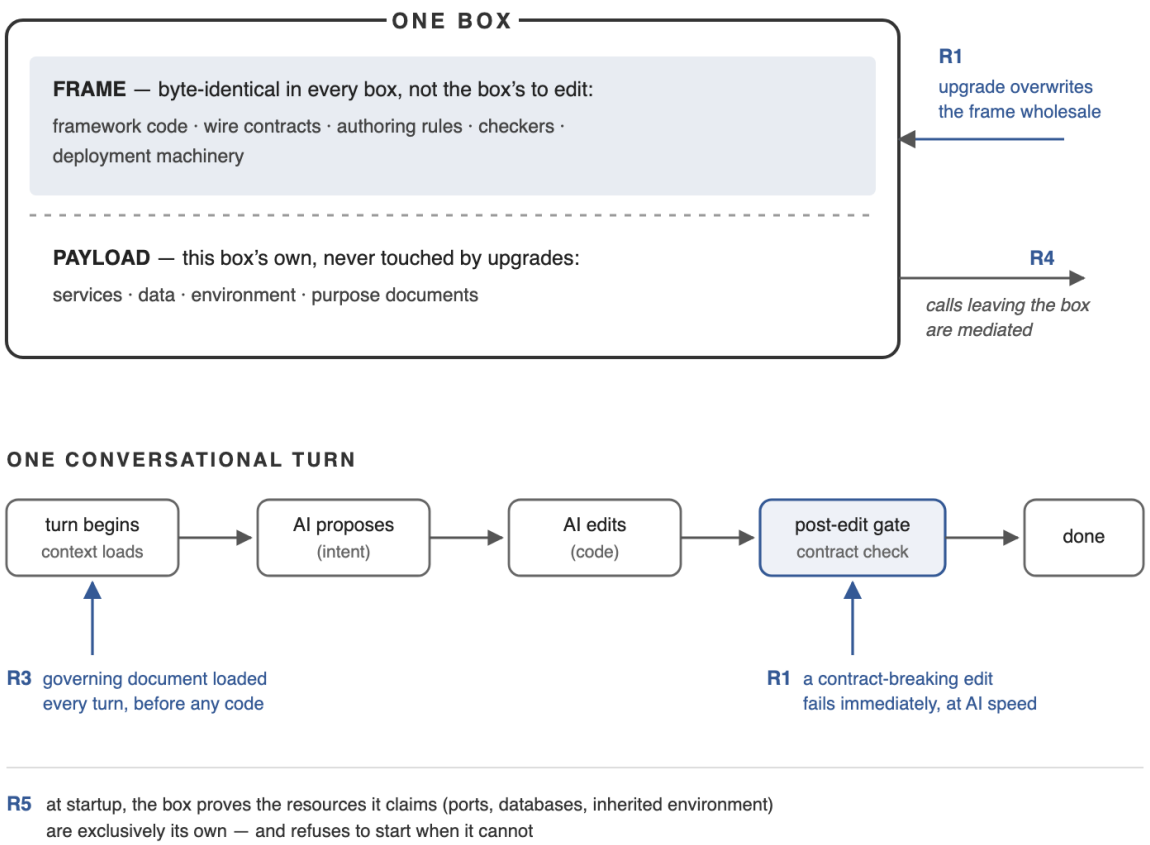


*Figure 1: anatomy of a box. The horizontal split is the pattern's only structural boundary; the arrows mark where each requirement takes effect. R3 acts before any code exists, R1's gate acts at edit time, R5 at startup — three different moments, which is why none of them substitutes for another.*

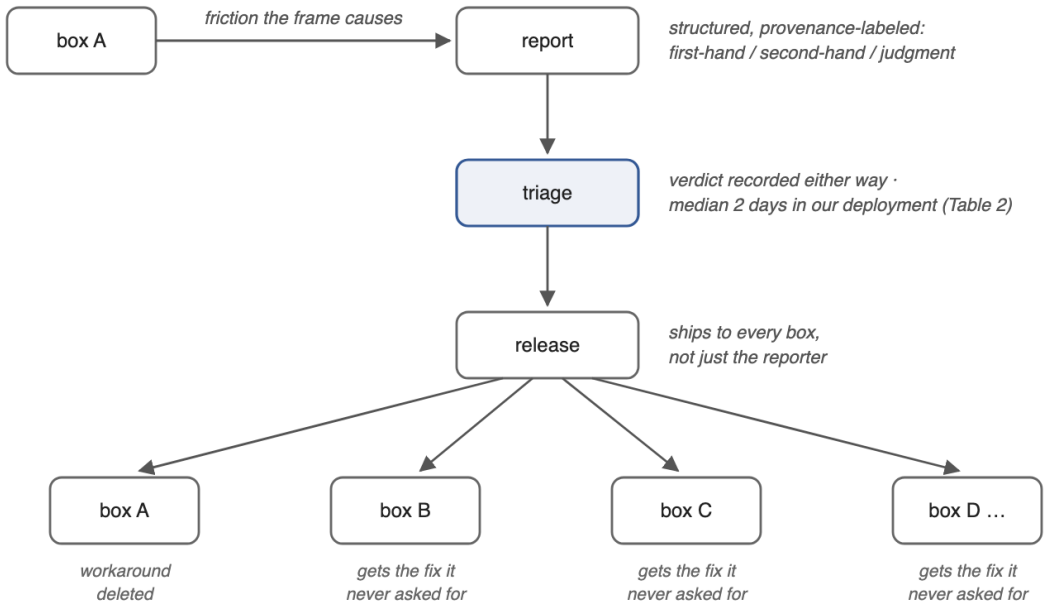


*Figure 2: the upstream-first loop (R2). The asymmetry is the point: friction is felt by one box and repaid to all of them. One release in our window closed six reports originating from three different boxes (§4.1).*

### 2.3 What invariance buys

The claimed benefits. Four are measured in §4; the first is the pattern's central promise and, as we flag below and in §7, the one we can argue for but have not measured:

- **Mutual intelligibility at any N** *(claimed, not measured)*. Because the frame is identical everywhere, every box stays legible to anyone — human or AI — who knows the standard. This is the property golden paths aim at and, being advisory, cannot guarantee (§6). Our evidence for it is indirect: upstream fixes land across boxes without per-box translation, and one release routinely closes reports from several different boxes (§4.1). A direct test — measuring what it costs a person or an AI to become productive in a box they have never seen — is the experiment this paper does not report, and the first we would run next.
- **Compounding evolution.** R2 turns one box's friction into every box's improvement: the standard absorbs its instances' lessons release by release (§4.1), instead of each box absorbing them alone.
- **No silent forks.** Divergence is deleted by upgrade (R1), resolved upstream (R2), or carried as enumerated debt — never invisible (§4.2).
- **Governable AIs at AI speed.** R1's automatic consequence and R3's per-turn window are the only two points where an AI's hundreds of decisions per hour can actually be constrained (§4.1, §4.3); review-time correction cannot keep up.
- **Ownership beyond programmers.** A standardized box with a per-turn governing document can be *driven* by a non-technical owner from day one (§4.4).

### 2.4 What invariance costs

The costs are structural, not incidental; an adopter should price them in. Each is measured or bounded in §4:

- **Version lag.** Boxes trail the standard's head by days to months, and work that needs a new frame capability must upgrade first — the alternative, editing the frame, is exactly what the model forbids (§4.2). Lag is the honest, visible price of the invisible forks it replaces.
- **A central triage bottleneck.** R2 concentrates evolution in the standard's maintainer. At our scale the loop is fast (§4.1); how triage scales to fifty boxes is unknown (§4.5).
- **Provisioning burden.** Instantiating a box requires judgment its future owner may not have; until provisioning is slimmed to delivery, every non-technical owner costs an expert a session (§4.4).
- **Error amplification.** A gap or mistake in the standard replicates into every box at once; the 5-of-5 episode (§4.3) is exactly this failure observed — the scaffold's missing artifact became every box's missing artifact. R2 is the mitigation, not a cure: the same replication that spreads the error also makes it visible in every instance.
- **Reduced local freedom.** Work blocked by the frame waits for upstream or carries marked debt; a box cannot simply take its own path — the box that needed a new framework switch had to upgrade its whole bundle first rather than edit one launcher line (§4.2). This is the point of the pattern — and still a real cost to the box that is blocked today.

### 2.5 Why now: the pattern's capability threshold

The container model is not timeless advice: it becomes meaningful only past a threshold of AI capability, and the argument is two-sided — the same capability that makes the box *affordable* is what makes it *necessary*.

**Below the threshold, the box is unaffordable.** The unit only works if a complete production stack can be built and operated by one box's crew — in the limit, one person with an assistant, or no humans in the daily loop at all (§2.1). Before coding assistants crossed that line, a full stack per role meant a team per role: the unit degenerates back into teams sharing systems, and "give marketing its own stack" is an absurdity rather than an option. The same capability also collapses the cost of *following* the standard: the frame's contracts and authoring rules are consumed primarily by AIs, so every new box's AI arrives already conversant with the standard — and onboarding cost, the historical reason organizations share one system instead of federating many, approaches zero.

**Below the threshold, the box is also unnecessary.** When humans made the architectural decisions, divergence happened at human speed, and review-time governance approximately held: a style guide plus code review could keep a small organization coherent. The risk this paper opens with — N locally optimizing AIs producing N mutually alien architectures — is a capability-era risk: hundreds of decisions per hour, each locally reasonable, none reviewed. And construction ability is destruction ability: an AI capable of building a full stack is equally capable of quietly rearchitecting one, "improving" the framework in place, or corrupting a neighbor's database through a shared port. Advisory governance fails exactly when the governed party outpaces the governor.

The box is therefore sized to the AI, in both directions. Inward, it is the largest scope one box's crew can genuinely own: a whole stack. Outward, it is the containment vessel that bounds the blast radius of any single AI failure: the frame is physically not editable (R1), reach beyond the box is mediated or absent (R4), shared infrastructure must be proven one's own before it is touched (R5), and intent is governed before code (R3), so the worst case of a misbehaving AI is one box's payload, not the organization. That boundary must itself be enforced rather than assumed — R5 exists precisely because we assumed it once and two boxes quietly shared a database for it (§4.1, L3). The shipping container carries the same lesson [3]: standardized boxes made no economic sense while cargo was handled by hand; the container is cargo sized to machines — cranes, cells, chassis — and it repaid its constraints only once mechanized handling existed. The box is software sized to AI handling, and it inherits both properties: below the capability threshold it is bureaucracy; above it, it is the difference between an organization and a blast zone.

A corollary: the pattern's relevance scales with model capability. Better AIs make a box cheaper to fill and an unbounded jurisdiction more dangerous, so we expect the §2.3–§2.4 trade to improve, not decay, as models improve (see also §7, "confounded timeline").

### 2.6 Scope and non-goals

The pattern targets organizations where each role owns a full stack and the stacks must stay mutually governable; the setting we have measured is one small organization within a single trust domain. It is *not* OS-level containerization (the invariant artifact is an organizational standard, not a runtime image); it is *not* an all-AI software company (humans are retained as each node's accountable owners, however automated daily operation becomes); and it is *not* a golden path (the standard is enforced by consequence, not recommended by a platform team). §6 develops these distinctions.

## 3. SOLO: One Implementation

SOLO is an open-source, AI-native microservice framework (Node.js / Express / Redis): 13 services behind a single Ed25519-signing router with method-level permissions, an entity factory with audit trails and write-ahead logs, a declarative fulfillment engine, and an orchestrator with human approval gates and m-of-n signatures. This section describes only the mechanisms that realize §2's requirements; the general architecture is documented in the repository.

| §2 requirement | SOLO mechanism |
|---|---|
| Box as unit (§2.1) | scaffold + single-file versioned bundle (§3.1) |
| R1 enforced frame | `[Solo]`/`[Project]` zones, overwrite-on-upgrade, divergence detection (§3.2); post-edit contract gate (§3.3) |
| R2 upstream-first evolution | dual feedback channels, human and machine (§3.4) |
| R3 per-turn governance | root governing document + guardrail skill (§3.5) |
| R4 mediated interaction | router-only calls within a box; bridge mesh across boxes — designed, not deployed (§3.6, §4.5) |
| R5 proven resource claims | launcher proves Redis ownership and fail-fast port claiming, both fail closed (§3.6) |

*Table 1: each pattern requirement and the SOLO mechanism that realizes it. Only R4 is partly unrealized.*

### 3.1 The box as artifact: scaffold and bundle

A box is instantiated by a scaffold script from a **single-file framework bundle** (`solo.v{X}.js`) plus shared source directories (`library/`, `sample/`, `autocheck/`), contract documentation (`docs/authoring/`), an AI guardrail skill, and deployment scripts — all marked `[Solo]`: this is the frame. The bundle is port-agnostic: a per-box services manifest decides which services activate on which ports, so one immutable artifact serves every box. Everything created inside the box — private services under `api/apps/`, environment, seeds, the operator UI — is `[Solo]`-free and never touched by upgrades: this is the payload. The stack is multi-actor by construction, realizing §2.1's n-human, n-AI unit: a user service manages accounts, sessions and permits for any number of humans, and any number of AI actors can work the box — interactive coding sessions and scheduled collection agents operating on the stack directly, event-subscribed sentinels reacting to its events, and external agents that bootstrap through the router's self-describing guide or its MCP adapter (§3.4).

### 3.2 Upgrade semantics and divergence detection (R1)

`upgrade.sh` replaces the read-only zone wholesale (deletions propagate) and leaves `[Project]` files alone. For the gray zone — deployment scripts that boxes legitimately customize — it compares the installed file against the *previous* release's stock version: if unmodified, it is upgraded silently; if modified, it is **not** overwritten, the new stock version is staged alongside as `<name>.solo-{ver}.new`, and the upgrade report flags `DIVERGED`. Divergence is thus never silent: it is either resolved by merging or carried as visible, enumerated debt.

The mechanics have engineering antecedents we build on deliberately: package managers already give dependency code overwrite-on-upgrade semantics, template updaters such as cruft and copier [5] propagate scaffold changes into derived projects with drift detection, and enterprise "clean core" doctrine draws the same modified-core-versus-extension line for ERP upgrades [6] (§6). SOLO extends the overwritten zone beyond code — to contract documentation, deployment scripts, and the AI guardrail skills, all in-tree — and reframes the semantics from a convenience (staying current with a template) to a governance boundary (the standard is physically not the box's to edit).

### 3.3 The gate in the AI editing loop (R1, at AI speed)

A static contract checker (`autocheck`) verifies naming (`{service}.{entity}.{action}`), wire-contract conformance, declaration/registration consistency, and red-line rules (services must not call each other directly; all calls go through the router). Crucially, it is wired into every AI's tool loop as a post-edit hook: an AI cannot finish an edit that breaks the contract without seeing the failure immediately. This meets R1's speed clause: the standard is checked at the speed the AIs work, not at review time.

### 3.4 Two feedback channels (R2)

The upstream-first loop has a human channel and a machine channel:

- **Human channel:** structured markdown reports in the framework repo's `docs/feedback/`, written by a box's humans after real friction; triage moves them to `done/` with a recorded verdict, whether or not the proposal was accepted.

- **Machine channel (`system.report`):** any AI agent operating against a box can *anonymously* file a "the system cannot do X" report through the router at the moment a task hits a wall. Reports are deduplicated; repeated collisions increment a counter, so triage priority is literally "how many tasks hit this wall." The router's self-describing guide teaches external agents that this channel exists.

### 3.5 Per-turn governance artifacts (R3)

Each box carries a root governing document (in our deployment, the AI harness's `CLAUDE.md` convention) auto-loaded every turn, and a guardrail skill that triggers on service edits. These are distinct instruments: the skill governs *how code is written*; the root document governs *what should exist at all* — and, as §4.3 shows, the two are not substitutes.

### 3.6 Mediation and resource claims (R4, R5)

Within a box, services are forbidden to call each other directly — every call goes through the signing router, and the §3.3 gate enforces the rule statically. Across boxes, the coordination layer is specified as a cryptographically authenticated bridge mesh with narrow permits, its configuration changes routed through the framework's m-of-n approval chain; it is designed and adversarially reviewed but **not deployed** (§4.5). R4 is therefore realized today only in its intra-box half; cross-box questions are answered by humans.

R5 is realized in the launcher, and only after the incident that forced it (§4.1): before binding, a box proves that the Redis instance on its configured port is its own rather than merely reachable, and refuses to start otherwise; front-end port claiming likewise fails closed rather than warning and continuing. Both checks are in the frame, so every box inherits them on upgrade — the requirement and its enforcement travel together.

## 4. Experience and Evidence

**Setting.** One maintainer develops the framework; seven long-running derived systems (anonymized here as Projects A–G) are built and operated on scaffolded boxes — plus an eighth box provisioned as a one-off trial for a non-technical role (the §4.4 case study) — spanning organizational dashboards, financial tracking, trend collection, job orchestration, and an ERP. Box populations vary and overlap rather than forming one-to-one pairs: most boxes share a single human operator, who owns several boxes at once; individual boxes are worked by multiple AI actors (interactive sessions, scheduled collectors, external agents arriving through the router's self-describing guide, §3.4); boxes with automated roles run most days with no human in the loop, their humans appearing only as owner and occasional maintainer; and the §4.4 box involved two humans in different roles, a provisioner and an operator. The deployment topology spans a home server, two VPSs, and developer machines. The framework's public repository (github.com/ff13dfly/solo) opened in July 2026; the observation window for the statistics below is June 14 – August 24, 2026 unless stated otherwise.

### 4.1 The loop runs, and it converges upstream (R2; compounding evolution)

Over the ten-week window the framework cut **21 tagged releases** (v1.1.0 → v1.2.2). The feedback corpus holds **31 structured reports**, of which **23 are triaged and archived** with recorded verdicts and 8 are

pending. Because archiving a report is a file move recorded in version control, the loop's cadence is auditable rather than self-reported (Table 2).

| Measure (June 14 – Aug 24, 2026) | Value |
|---|---|
| Tagged framework releases | 21 (v1.1.0 → v1.2.2) |
| Structured reports filed | 31 |
| Reports triaged and archived | 23 (8 pending) |
| Median report → triage latency | **2 days** (n = 21 of 23; range 1–19) |
| Archived reports cited by filename in release notes | 14 of 23 (lower bound; others credited in prose) |
| Boxes that filed at least one report | 6 of 7 |
| Reports from the most active single box | 15 of 28 attributable (3 reports name no box) |

*Table 2: the upstream loop, measured from repository history.*

Two caveats keep these honest. The latency figure omits two archived reports that first appear in version control already inside `done/`, so their pre-archive life is unobservable; and triage arrives in batches rather than continuously, so the median describes the loop's typical turnaround, not a steady rhythm. The filename-citation count is likewise a floor: the `system.guide` case below shipped in v1.1.11 whose notes credit the originating box in prose without naming the file.

Releases are traceable to reports; three cases illustrate the loop's shape:

- **An agent-facing gap found by agents.** A derived project (Project G) reported that external AI agents could not bootstrap against a box without human hand-holding. Triage produced the framework's self-describing `system.guide` mechanism — anonymous first-call returns the authentication flow, envelope format and error codes — shipped in v1.1.11. The report that motivated it is archived verbatim.
- **A silent-failure class found by co-located boxes.** Two boxes sharing a machine (Projects D and F) discovered that a Redis port collision fails *silently*: the second box attaches to the first box's database and corrupts it. The report traced the root cause and the fix class — startup checks must prove ownership, not just reachability, and must fail closed. The framework's launcher now verifies Redis ownership and refuses to start on another box's port; the same release hardened front-end port claiming from warn-and-continue to fail-fast.
- **Batch upcycling.** v1.1.16 closed out six derived-project reports in one release — and those six came from **three different boxes**, so one upgrade repaid friction that three separate crews had hit independently; v1.1.17 closed five more from a single project (Project A). Upcycling is routine, not exceptional: it is the release train's main cargo, and it is the mechanism behind the compounding-evolution claim of §2.3.

Two observations. First, the reports' **evidence-provenance discipline** (R2) earned its keep: triage caught at least one case where a derived project's second-hand claim about router behavior was wrong (the router had returned the correct count; an intermediate layer dropped it), which a first-hand/second-hand label made cheap to detect. Second, the loop's cost asymmetry matters: a report costs its author an hour; a silent divergence costs the maintainer an unbounded amount later. The protocol prices this correctly.

### 4.2 The lag cost, measured (§2.4)

A snapshot across boxes (2026-08-15) shows bundle versions spanning v1.0.0 to v1.1.15: boxes lag the standard's head by days to months. Under the container model this is not drift — every box is on *some* exact release of the same standard, divergence is enumerated (§3.2), and the upgrade path is one file copy plus staged merges. But lag is real cost: a box on an old bundle cannot use new framework switches (a derived project needed the framework's Redis-password support and had to upgrade its bundle first, because the alternative — editing the read-only launcher — is exactly what the model forbids). We consider this trade explicit and correct: the alternative to visible lag is invisible fork.

Lag is affordable for a further reason, and it is a flexibility the invariance buys back: version skew does not fracture communication. The wire contract — envelope, signing, method naming, error codes — lives in the frame and evolves additively in the current major version, so a client that speaks the standard can talk to a box on any release in the span, and each box chooses *when* to upgrade without losing the ability to be talked to — the per-unit upgrade autonomy that ERP fleets lose to upgrade paralysis (§6). The discipline held with one deliberate exception in our window: a release that made two authentication parameters mandatory, shipped as a flagged breaking change. Even that exception travels with its contract: the upgrade tool scans every release note newer than the box's installed version and raises any required downstream action as a banner, so a lagging box learns what its jump will cost before it jumps.

### 4.3 A natural experiment: 5 of 5 boxes re-invented the same missing artifact (R3; error amplification)

The scaffold initially shipped the guardrail skill (§3.5) but **no root governing document**. All five derived projects observed at the time had independently written one by hand. Five out of five is not preference; it is the standard revealing its own gap through its instances — the §2.4 amplification cost observed in the wild, and precisely the signal the R2 loop exists to carry: it was duly reported upstream with a proposal to ship a template. The episode also demonstrated R3's distinction empirically: the skill (which triggers on service edits) could not have prevented any of the failure modes the root documents were written to stop — feature over-proposal, speculative schema creation, unexported irreproducible data — because those happen before any service file is edited.

### 4.4 Provisioning a box for a human with no terminal experience (§2.3, §2.4)

The model's promise is that *every* role can own a box. We tested the worst case: provisioning a box for a marketing colleague who had never used a terminal. Result: feasible, with a sharp division found in practice — the colleague could *operate* the box (converse with an AI assistant; the per-turn governing document did the day-to-day governing), but could not *provision* it: installing the runtime dependencies, allocating ports, generating keys and validating Redis ownership all required judgment the colleague could not supply. The working solution was to generate the complete box on an expert's machine and deliver it ready to run. This works but does not scale (every new person costs an expert a session), which converts "deployment slimming" from an optimization into a feasibility precondition for the model at organization scale — a prioritization insight we fed back into the roadmap. The episode also showed the box, not a fixed human–AI pairing, to be the durable unit: this box was provisioned by one human and is operated day to day by another, and it changed hands without changing its frame.

### 4.5 What the model has *not* yet demonstrated

Honesty requires listing the unproven half. R4's cross-box half — the bridge mesh with narrow permits — is designed and adversarially reviewed but not deployed; all cross-box questions today are answered by humans. (As of the final revision of this paper, late August 2026, a first same-operator testbed — a loopback bridge between two co-located boxes — and a three-channel asynchronous interaction protocol for it, downlink archival-acknowledgment plus periodic pull plus doorbell, have entered specification; deployment has not begun.) Governance of the *coordinator* itself — who may change the upstream's permits, under what approval — is designed (routed through the framework's m-of-n approval chain) but likewise undeployed. And the model has run under one maintainer; we do not know how triage scales when reports arrive from fifty boxes rather than seven (§2.4's bottleneck cost, unbounded above our scale).

## 5. Lessons

**L1 — A standard is a consequence, not a document.** Every mechanism that actually held (read-only zone, fail-fast port claims, post-edit contract gate) works because violating it costs something automatically. Every mechanism that was advisory (docs describing conventions) was violated by default, at AI speed. If a rule matters, wire a consequence to it; if you cannot, expect divergence.

**L2 — Govern AIs per-turn, not per-edit.** The damaging failure modes happen before code: proposing scope, inventing schemas, choosing what to persist. Only an always-loaded governing document reaches that window; edit-triggered guardrails structurally cannot (§4.3).

**L3 — Silent failure is the default failure mode of co-located boxes.** Port collisions, inherited environment variables, wrong-database attachments: none of these crash; all corrupt. Startup checks must prove *ownership* and fail closed. We now treat "warn and continue" in a launcher as a bug class.

**L4 — Label evidence provenance in feedback.** Requiring reports to mark each claim first-hand / second-hand / judgment made bad upstreaming cheap to catch (§4.1) and kept the standard's evolution anchored to measurements rather than hearsay.

**L5 — Version lag is the honest price of no-fork.** Enforced invariance converts what would be N invisible forks into N visible lags plus an enumerated divergence list. Buy it knowingly: the lag is real, bounded, and repayable; the forks are none of those.

**L6 — For non-technical owners, provisioning is the wall, not operation.** With a per-turn governing document, a non-programmer can safely *drive* a box on day one. Getting them a box is the hard part, and it is an infrastructure problem (dependency slimming, prebuilt delivery), not a prompting problem (§4.4).

## 6. Related Work

**All-AI software organizations.** MetaGPT [7], ChatDev [8], AgentMesh [9] and CodePori [10] assign organizational roles to LLM agents and automate the SDLC end to end. The container model inverts the premise: humans are not simulated but retained as each node's accountable owners, and the replicated artifact is the infrastructure standard, not the org chart.

**Human–AI teaming.** The HCI and organizational literature studies task-level collaboration: trust [11], situation awareness [12], team design and reviews [13,14]. It treats the infrastructure the humans and AIs work *inside* as given; the container model is precisely about that infrastructure.

**Platform engineering and golden paths.** Industry practice equips engineers — and recently agents [1,2] — with paved roads inside one organization's platform. Recent academic treatments frame skills and policies as agent-consumable institutional knowledge [15,16]. The closest of these, Knowledge Activation [16], converts institutional knowledge into an agent-traversable graph of atomic units and reports developer-experience gains from a single-organization deployment; it standardizes the *knowledge schema* consumed by agents, but the stack remains centrally operated — there is no per-unit ownership, no enforced-invariance mechanism, and no documented protocol by which the units' friction evolves the standard. Golden paths generally are advisory and centrally operated; the container model's standard is enforced by overwrite semantics and is designed for federated units that each own a full stack.

**Enterprise platforms: ERP flexibility and product lines.** The tension the container model resolves — one standard, many owners who need local variation — is decades old in enterprise software, where its lesson was learned as *upgrade paralysis*: ERP customers who modified the vendor core found every upgrade priced by their own divergence, and the industry's mature answer is doctrine the container model would recognize. SAP's "clean core" mandates that extensions live outside an unmodified core, attached only through released extension points, precisely so that extensions do not break upgrades and upgrades do not break extensions [6]; software product lines engineer the same discipline academically, as a shared platform with designed variation points [17]. The container model shares the invariant-core/owned-extension split (§2.1) but differs on three axes. *Granularity*: the standardized artifact is a complete per-role stack inside one small organization, not one enterprise-wide monolith. *Kind of flexibility*: ERP-style flexible deployment is anticipated variation — configuration knobs and extension points the vendor designed in advance — whereas a box's payload is open variation: any service its crew can build, a freedom that became governable only when AI made the frame's contracts checkable at editing speed (§2.5, §3.3). *Evolution*: an ERP customer's friction enters a vendor's opaque enhancement pipeline; a box's friction enters an in-repo, provenance-labeled triage record that every crew can read (R2). Fittingly, one of our seven boxes is itself a small ERP built as payload on the standard frame (§4) — the model subsumes the use case rather than competing with it. And where enterprise software reached clean-core doctrine after decades of divergence pain, the container model starts there: the frame is born read-only.

**Organizational antecedents.** Enforced standards across autonomous units are not new to organizations. Amazon's 2002 service-interface mandate — all teams expose functionality only through service interfaces, on pain of dismissal, as later documented by Yegge [18] — is enforced invariance for team boundaries; Haier's *rendanheyi* model decomposes the firm into thousands of self-managing microenterprises on a shared platform [19]. The container model can be read as the human–AI-era descendant of both, with two deltas: the autonomous unit shrinks to a single box — small enough for one person to own, with no fixed bound on the humans and AIs working it — and the enforcement mechanism moves from managerial policy into the filesystem and upgrade semantics of the unit's own stack.

**Agent context files.** A recent empirical line studies the governing documents themselves: large-scale characterizations of AGENTS.md/CLAUDE.md files find them to be complex, config-like artifacts dominated by build and architecture content [20,21], and controlled evaluations of their effect on task

success report mixed results [22]. That literature measures the files as they are used in the wild — largely as *technical* context. Our §4.3 observation is complementary and different in kind: when boxes lacked a root governing document, every box independently created one, and what they put in it was not build context but *organizational* governance (purpose, scope discipline, data policy) — content the task-success lens does not measure.

**Runtime agent governance.** Governance-as-a-Service [23], Institutional AI [24], and sovereign-agent infrastructure [25] govern agent *behavior* at runtime with monitors, scores, and sanctions. The container model governs the *substrate* statically and cheaply — filesystem semantics and upgrade overwrites — and reserves runtime governance (approval gates, signatures) for the actions that need it.

**End-user software engineering.** The vision of non-programmers owning their software is classic [26] and newly practical with LLMs. Our contribution to that line is narrow and empirical: the binding constraint we observed is provisioning, not operation (§4.4, L6).

**Separating the pattern from its neighbours.** Each line above shares something with the container model, which invites the reasonable objection that the model is existing practice under a new metaphor. Table 3 states the differences on the axes that matter, and shows that no neighbour combines them: several enforce a standard, several grant local ownership, one runs an upstream protocol, and none governs AIs per-turn or federates units that each own a complete stack.

| Approach | Unit | Standard enforced? | Unit owns a full stack | Documented upstream evolution | AI governed per-turn | Federated units |
|---|---|---|---|---|---|---|
| Golden paths [1,2] | team / service | No — advisory | No — platform-operated | Informal | No | No |
| InnerSource [4] | project | No | Partly | **Yes** — its core practice | No | Partly |
| Software product lines [17] | family member | Yes — variation points | Only within designed variation | Via platform team | No | No |
| Clean core [6] | enterprise system | Yes — upgrade-safe extension points | Extensions only | Vendor-driven, opaque | No | No |
| Knowledge Activation [16] | knowledge unit | No — schema only | No — centrally operated | Not documented | Partly (agent context) | No |
| All-AI organizations [7–10] | agent role | n/a | No human owner | n/a | Yes (prompts) | No |
| **Container model** | **box** | **Yes — overwrite-on-upgrade** | **Yes** | **Yes — triaged, provenance-labeled** | **Yes** | **By design (R4); intra-box only today (§4.5)** |

*Table 3: the container model against adjacent approaches. The final cell is deliberately not a "yes": cross-box federation is specified and reviewed but undeployed, so we claim it as a requirement, not a result.*

To our knowledge, the specific combination — the box as the organizational unit binding humans and AIs, standard-with-consequences via a read-only zone, upstream-first evolution fed partly by the AIs

themselves, per-turn AI governance, and proven resource claims between co-located units — has not been described or evaluated in the literature. The individual mechanisms are mostly not new, and §3.2 names the ones we build on directly; the contribution is the combination and the measured account of running it.

## 7. Threats to Validity

**Single organization, single maintainer.** All seven boxes and the framework share one operator-culture; effects may not transfer. The maintainer is also this paper's author: selection and confirmation bias are live risks, which we mitigated only partially by anchoring every claim in §4 to written artifacts (feedback reports, changelogs, tagged releases) that predate the paper.

**Single implementation.** The pattern (§2) and the implementation (§3) are observed only together; we cannot separate the pattern's merits from SOLO's particular design quality. A second, independent implementation of §2's requirements is the obvious next test.

**Small N, no control.** Seven boxes, five in the §4.3 observation; no baseline organization running the same projects without the container model. The 5/5 result is suggestive, not statistical.

**Skewed evidence base.** The feedback corpus is dominated by one box (15 of 28 attributable reports, Table 2), and one of the seven filed none at all. The loop's measured cadence therefore describes the standard's relationship with its most active instance more than with its median one.

**The central benefit is argued, not measured.** Mutual intelligibility (§2.3) is the pattern's main promise and the one claim §4 does not test; our support for it is indirect (cross-box fixes landing without translation). A box-onboarding cost experiment would settle it and we have not run one.

**The capability threshold is not falsifiable here.** §2.5 argues the pattern only pays above a level of AI capability, but our whole observation window sits above that level; we have no below-threshold arm, so that argument rests on the historical analogy and on mechanism, not on data.

**Confounded timeline.** The observation window coincides with rapid improvement in the underlying AI models; some outcomes attributed to the model may partly reflect better AIs.

**Self-reported metrics.** Release and feedback counts are from the repository and are auditable; qualitative claims (e.g., "silent failure is the default failure mode") generalize from a small incident set.

## 8. Conclusion

The container model treats the scaling of AI-assisted development as a standardization problem and borrows the shipping container's answer: make the standard physically consequential, evolve it only upstream, and let value come from invariance. The pattern is deliberately priced: it aims at mutual intelligibility — the promise we argue for but have not yet measured — and it demonstrably buys compounding evolution and the absence of silent forks, paying in version lag, a triage bottleneck, provisioning burden, and amplified standard errors. Ten weeks and seven boxes into one implementation, the loop runs: the standard absorbs its instances' lessons at a cadence of roughly two releases a week, divergence is visible instead of silent, and a non-programmer can drive a box on day one. The unproven half — federation and coordinator governance — is where our work goes next. We offer the pattern, one

implementation, and our ledger of costs as a starting point for others building organizations out of human–AI boxes.

## Acknowledgments and AI Disclosure

In keeping with the subject of this paper, the manuscript itself was produced inside the model it describes: the text was drafted by a generative AI assistant (Claude, Anthropic) working under the author's direction inside one of the boxes described in §4, from the repository's feedback reports, changelogs, and release history. The system design, all measurements, the feedback corpus, and all judgments and conclusions are the author's; the author reviewed and verified every claim against the primary artifacts and takes full responsibility for the content. This disclosure follows the ACM and IEEE policies on the use of generative AI in publications.

## Data Availability

The framework, its release history (git tags v1.1.0 – v1.2.2), the complete feedback corpus (`docs/feedback/`, including archived reports with their recorded triage verdicts), and the changelogs cited in §4 are public at https://github.com/ff13dfly/solo. Every quantitative claim in Table 2 is recomputable from that repository's version-control history. The derived projects' payloads are private; the paper refers to them by anonymized label (Projects A–G). The archived feedback reports in the public corpus retain the projects' working names, so the anonymization is presentational rather than cryptographic.

## References

[1] Platform Engineering for the Agentic AI Era. Microsoft All Things Azure blog, 2026. (industry reference) https://devblogs.microsoft.com/all-things-azure/platform-engineering-for-the-agentic-ai-era/ (accessed August 2026).

[2] AI Agents Need Platform Engineering, Too. platformengineering.com, 2026. (industry reference) https://platformengineering.com/features/ai-agents-need-platform-engineering-too/ (accessed August 2026).

[3] M. Levinson. The Box: How the Shipping Container Made the World Smaller and the World Economy Bigger. Princeton University Press, 2006.

[4] K.-J. Stol and B. Fitzgerald. Inner Source — Adopting Open Source Development Practices in Organizations: A Tutorial. IEEE Software, 32(4), pp. 60–67, 2015. doi: 10.1109/MS.2014.77.

[5] cruft and copier: project-template update tools with drift detection. (industry reference) https://cruft.github.io/cruft/ and https://copier.readthedocs.io/ (accessed August 2026).

[6] SAP. "Clean Core" extensibility guidance for SAP S/4HANA Cloud — extensions must not break upgrades and upgrades must not break extensions; extend via released extension points on-stack or side-by-side on SAP BTP. SAP News Center / SAP Community, 2023–2025. (industry reference) https://news.sap.com/2025/08/extend-sap-s4hana-cloud-right-way-clean-clear/ (accessed August 2026).

[7] S. Hong et al. MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework. ICLR 2024. arXiv:2308.00352.

[8] C. Qian et al. ChatDev: Communicative Agents for Software Development. ACL 2024. arXiv:2307.07924.

[9] S. Khanzadeh. AgentMesh: A Cooperative Multi-Agent Generative AI Framework for Software Development Automation. arXiv:2507.19902.

[10] Z. Rasheed et al. CodePori: Large-Scale System for Autonomous Software Development Using Multi-Agent Technology. arXiv:2402.01411.

[11] M. J. McGrath et al. Collaborative Human-AI Trust (CHAI-T): A Process Framework for Active Management of Trust in Human-AI Collaboration. Computers in Human Behavior: Artificial Humans, 2025.

[12] Q. Gao, W. Xu, M. Shen, and Z. Gao. Agent Teaming Situation Awareness (ATSA): A Situation Awareness Framework for Human-AI Teaming. arXiv:2308.16785.

[13] T. O'Neill, N. McNeese, A. Barron, B. Schelble. Human–Autonomy Teaming: A Review and Analysis of the Empirical Literature. Human Factors, 2022.

[14] B. Lou, T. Lu, T. S. Raghu, and Y. Zhang. Unraveling Human–AI Teaming: A Review and Outlook. arXiv:2504.05755.

[15] R. Britto, F. Palmgren, N. Saini, and M. Ohlin. The AI-Native Large-Scale Agile Software Development Manifesto. arXiv:2605.07717.

[16] G. Bakal. Knowledge Activation: AI Skills as the Institutional Knowledge Primitive for Agentic Software Development. arXiv:2603.14805.

[17] P. Clements and L. Northrop. Software Product Lines: Practices and Patterns. Addison-Wesley, 2001.

[18] S. Yegge. Stevey's Google Platforms Rant. Public post, 2011. (documents Amazon's 2002 service-interface mandate; the original memo is not publicly available)

[19] G. Hamel and M. Zanini. The End of Bureaucracy. Harvard Business Review, 96(6), Nov–Dec 2018. (Haier's rendanheyi microenterprise model)

[20] W. Chatlatanagulchai, H. Li, Y. Kashiwa, B. Reid, et al. Agent READMEs: An Empirical Study of Context Files for Agentic Coding. arXiv:2511.12884.

[21] W. Chatlatanagulchai, K. Thonglek, B. Reid, Y. Kashiwa, et al. On the Use of Agentic Coding Manifests: An Empirical Study of Claude Code. arXiv:2509.14744.

[22] T. Gloaguen, N. Mündler, M. Müller, V. Raychev, et al. Evaluating AGENTS.md: Are Repository-Level Context Files Helpful for Coding Agents? arXiv:2602.11988.

[23] S. Gaurav, J. Heikkonen, and J. Chaudhary. Governance-as-a-Service: A Multi-Agent Framework for AI System Compliance and Policy Enforcement. arXiv:2508.18765.

[24] F. Pierucci, M. Galisai, M. Syrnikov Bracale, M. Prandi, et al. Institutional AI: A Governance Framework for Distributional AGI Safety. arXiv:2601.10599.

[25] B. A. Hu and H. Rong. Sovereign Agents: Towards Infrastructural Sovereignty and Diffused Accountability in Decentralized AI. arXiv:2602.14951.

[26] A. J. Ko et al. The State of the Art in End-User Software Engineering. ACM Computing Surveys, 43(3), Article 21, pp. 1–44, 2011.